

PENQUEE ENGINEERING



Supreme Dashboard Developer Manual

Engineering, testing, deployment, and maintenance guide

IMPLEMENTATION GUIDE

For frontend engineers, API engineers, QA engineers, release engineers, and maintainers

Source-backed • Developer-only • Frontend application contract

Contents

Automatic table of contents

Chapter map

- 1. Purpose and scope
- 2. System overview
- 3. Technology baseline
- 4. Runtime architecture
- 5. Project structure
- 6. Prerequisites and installation
- 7. Configuration and environment variables
- 8. Commands and local workflow
- 9. Authentication and session lifecycle
- 10. Apollo transport, cache, and error handling
- 11. GraphQL operation map
- 12. Data flow and backend contract
- 13. Mounted modules and shared components
- 14. Unmounted components and unused operations
- 15. Testing topology
- 16. Build and deployment
- 17. Customization and extension workflow
- 18. Troubleshooting
- 19. Security and maintenance
- 20. Known limitations and technical debt
- Appendix A. Evidence baseline
- Appendix B. Release checklist

Document profile

Field	Value
Product	Supreme Dashboard
Audience	Engineering, testing, deployment, and maintenance teams
Application	React/TypeScript single-page administration client
Authority	Repository source, configuration, GraphQL documents, and tests
Companion	Separate Supreme Dashboard User Manual

Scope note: this edition describes implementation and operational dependencies. Step-by-step user training is intentionally maintained in the separate user manual.

1. Purpose and scope

Supreme Dashboard is a browser-based super-admin console for a multi-tenant platform. This repository contains the frontend only. Authentication, authorization, persistence, business validation, audit logging, subscription processing, refunds, and financial calculations ultimately belong to the external GraphQL service.

This manual covers:

- runtime composition and architectural boundaries;
- installation, configuration, commands, and developer workflow;
- authentication, session, Apollo, and GraphQL behavior;
- mounted pages, internal tabs, shared components, and operation ownership;
- automated test topology and targeted execution;
- static production builds and deployment requirements;
- safe extension, maintenance, security, and troubleshooting practices.

This manual does not define backend schema migrations, infrastructure provisioning, payment-provider behavior, or operator procedures beyond the feature map needed to understand the code.

1.1 Product feature map

Mounted destination	Engineering responsibility	Principal backend capability
Dashboard	Summary cards and status breakdown	Revenue/company/user/payment/expense aggregate
Plans	Plans and promo-code tabs	Plan and promotion queries/mutations
Companies	Company list, Company 360, payment history	Company, plan-assignment, refund, and payment actions
Subscriptions	Subscription/payment administration	Payment, due-date, suspension, resume, and manual actions
Users	Cross-tenant user administration and per-user activity	User list/update and activity-log query
Accounts	Infrastructure, expenses, monthly	Finance trend, expense, and report

Mounted destination	Engineering responsibility	Principal backend capability
	reports, annual reports	operations

The application does not use URL routes. AdminDashboard selects one of these destinations with React state.

2. System overview

2.1 System boundary

TEXT

Browser

```
Vite-built React application
├─ login/session shell
├─ admin navigation and pages
├─ Apollo Client
└─ export/print interactions
```

HTTP GraphQL
cookies + raw custom token

```
External GraphQL service
├─ authentication/authorization
├─ persistence and validation
├─ audit records
├─ payments/subscriptions
└─ finance and reporting
```

There is no backend, database, container definition, reverse-proxy configuration, or provider-specific deployment manifest in this project.

2.2 Runtime characteristics

- Client-rendered React 19 application.
- One browser entry point: `index.html` → `src/main.tsx`.
- One shared Apollo client.
- Authentication state stored in tab-scoped `sessionStorage`.
- Page navigation held in component state, not the URL.
- Browser-visible Vite configuration compiled into the bundle.
- Tables and dialogs combine backend query results with local UI filtering and form state.
- Mutations generally refetch their owning list after success.

3. Technology baseline

Concern	Implementation
Language	TypeScript targeting ES2022
UI runtime	React 19 and React DOM 19
Build/dev server	Vite 6
Styling	Tailwind CSS 4 through <code>@tailwindcss/vite</code> , plus local CSS
API client	Apollo Client 4 over HTTP GraphQL

Concern	Implementation
GraphQL runtime	graphql 16
Icons	Lucide React
Browser tests	Playwright
Component tests	Playwright experimental component testing for React
Static validation	TypeScript tsc --noEmit
Package format	ES modules ("type": "module")

Not configured:

- URL routing;
- ESLint or another style linter;
- an automatic formatter;
- a unit-test runner separate from Playwright;
- a global state-management library;
- code splitting at page boundaries;
- CI/CD workflows;
- container or infrastructure-as-code files.

`npm run lint` is a historical script name. It runs TypeScript validation, not a style linter.

4. Runtime architecture

4.1 Composition tree

TEXT

```

index.html
├── src/main.tsx
│   ├── React.StrictMode
│   │   └── GraphQLProvider
│   │       └── App
│   │           ├── unauthenticated login shell
│   │           └── AdminDashboard (authenticated)
│   │               ├── AdminLayout
│   │               │   └── current page selected in React state
│   │               └── Toast

```

Responsibilities:

- `src/main.tsx` mounts the React tree.
- `GraphQLProvider` exposes the singleton Apollo client.
- `App` owns login state, restoration, expiry, and logout.
- `AdminDashboard` owns the current top-level page and global toast state.
- `AdminLayout` owns the responsive shell, navigation, menus, focus behavior, and session-expiry countdown state; the current JSX does not render that value.
- Each page owns its query/mutation hooks, local filters, forms, detail panels, and reports.

4.2 Architectural boundaries

Boundary	Location	Responsibility
Entry/bootstrap	src/main.tsx	Root mount and provider composition
Authentication shell	src/App.tsx	Login, session restoration, expiry, logout
Transport	src/lib/apollo.ts	Endpoint, headers, credentials, cache, auth-error logout
GraphQL documents	src/lib/graphql.ts	Executable frontend query/mutation selection sets
Admin shell	src/components/admin/ AdminDashboard.tsx, AdminLayout.tsx	Page selection and responsive layout
Pages	src/components/admin/pages/	Feature-specific data and interactions
Shared UI	ColumnFilter.tsx, DateRangePicker.tsx, Toast.tsx	Reusable behavior
Utilities	src/lib/admin-utils.ts, useDebounceValue.ts	Formatting, conversions, normalization, debouncing
Backend	Outside repository	Security, persistence, authoritative rules, audit, payments

4.3 Navigation model

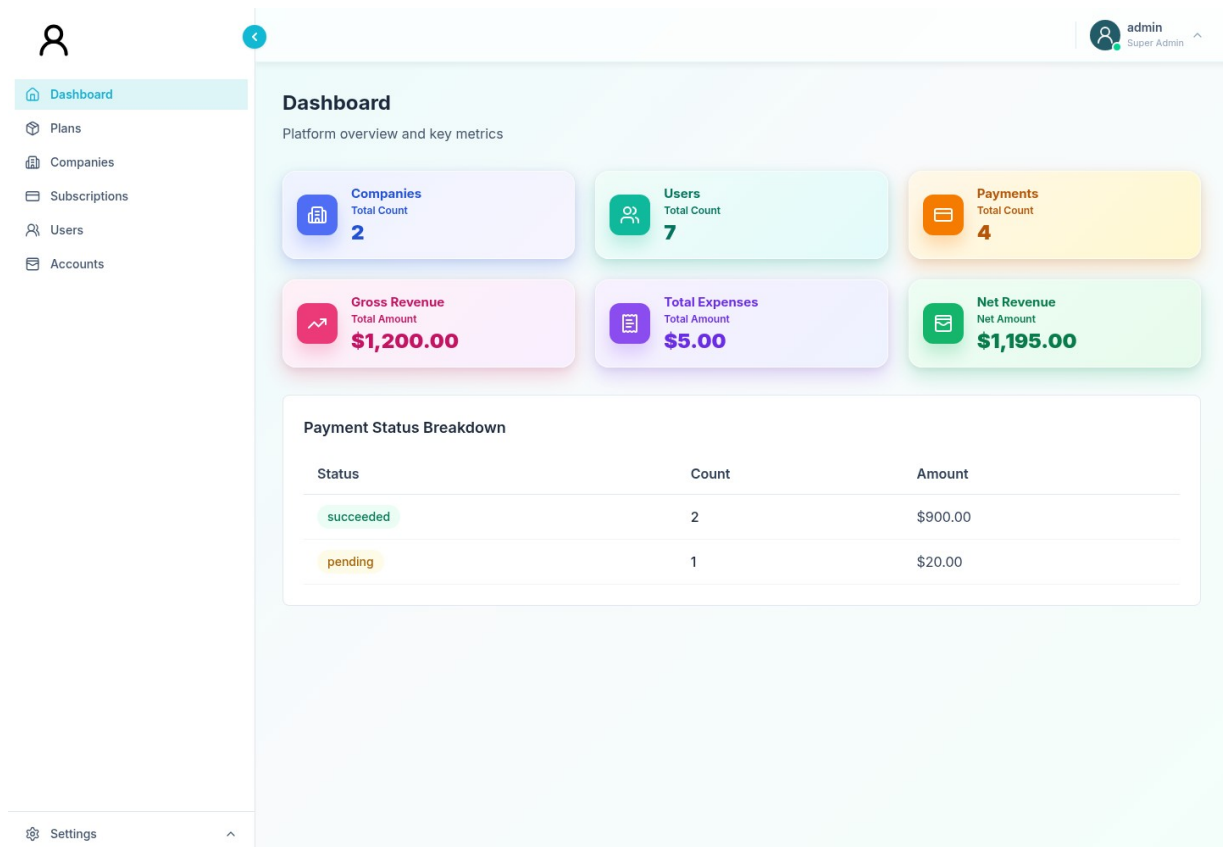


Figure 1. Desktop admin shell: `AdminLayout` navigation surrounding the mounted Dashboard page.

CurrentPage in AdminDashboard.tsx and the corresponding union in AdminLayout.tsx contain:

TEXT

dashboard | packages | companies | subscriptions | users | accounts

The visible label for packages is **Plans**. Changing a page updates in-memory state. Consequences:

- refresh returns an authenticated user to Dashboard;
- selected page cannot be bookmarked or deep-linked;
- browser Back/Forward does not traverse dashboard pages;
- current static hosting does not need SPA route rewrites beyond serving `index.html` at the root;
- the duplicated page union must be updated in both components when navigation changes.

4.4 State ownership

The project favors local state:

- App: credentials, authenticated username, login error, password visibility;
- AdminDashboard: active page and global toast;
- AdminLayout: menu, sidebar, focus, profile, and unrendered countdown state;
- pages: query variables, client filters, sort state, dialogs, forms, and selected records;
- Apollo: remote query results and mutation lifecycle.

No shared domain store exists. Before adding one, verify that state truly crosses page boundaries; otherwise keep ownership local.

5. Project structure

TEXT

```
supremedashboard/
├── public/assets/
│   ├── penqueue_logo.png
│   ├── penqueue_mascot.png
│   └── penqueue-favicon.svg
├── src/
│   ├── components/admin/
│   │   ├── pages/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── Plans.tsx
│   │   │   ├── Companies.tsx
│   │   │   ├── Subscriptions.tsx
│   │   │   ├── Users.tsx
│   │   │   ├── Accounts.tsx
│   │   │   ├── Infrastructure.tsx
│   │   │   ├── Expenses.tsx
│   │   │   └── UserActivityLog.tsx
│   │   ├── AdminDashboard.tsx
│   │   ├── AdminLayout.tsx
│   │   ├── ColumnFilter.tsx
│   │   ├── DateRangePicker.tsx
│   │   └── Toast.tsx
│   ├── lib/
│   │   ├── admin-utils.ts
│   │   ├── apollo.ts
│   │   ├── graphql.ts
│   │   └── useDebounceValue.ts
│   ├── providers/GraphQLProvider.tsx
│   ├── App.tsx
│   ├── index.css
│   └── main.tsx
├── tests/
│   ├── fixtures/
│   ├── integration/
│   └── feature folders
```

```

├── component-tests/
│   ├── fixtures/
│   ├── commerce/
│   ├── operations/
│   └── shared/
├── playwright/
├── docs/
├── index.html
├── package.json
├── package-lock.json
├── vite.config.ts
├── playwright.config.ts
├── playwright-ct.config.ts
└── tsconfig.json

```

Supporting contract notes:

- TESTING.md and tests/README.md describe test execution.
- backend-change-requests.md records requested backend capabilities; it is not backend code.
- src/expense-backend-agents.md proposes backend work tracks for the expense contract.

6. Prerequisites and installation

6.1 Required tools

- Node.js and npm.
- A maintained Node release compatible with Vite 6 and the committed lockfile. The project does not declare engines or pin a Node version.
- Access to a compatible GraphQL service for live use.
- An active super-admin account for live login.

Testing additionally requires:

- Google Chrome for E2E projects because playwright.config.ts selects the chrome channel;
- Playwright Chromium for React component tests.

6.2 Reproducible install

Run from the project directory:

```

SH
cd supremedashboard
npm ci
npx playwright install chrome chromium

```

Use `npm ci` for a lockfile-faithful install. Use `npm install` only when intentionally updating dependencies or `package-lock.json`.

On a clean Linux build agent, Playwright system libraries may also be required. `npx playwright install --with-deps` can install them but may need elevated privileges.

6.3 Minimal local startup

Create an uncommitted `.env.local`:

```

DOTENV
VITE_GRAPHQL_URI=http://localhost:8000/graphql/
VITE_GRAPHQL_DEBUG=1

```

Start the backend separately, then:

SH

npm run dev

The dashboard listens on `http://localhost:3001`.

7. Configuration and environment variables

7.1 Frontend variables

Variable	Required	Default	Read by	Notes
VITE_GRAPHQL_URI	No	<code>http://localhost:8000/graphql/</code>	<code>src/lib/apollo.ts</code>	Browser-visible GraphQL endpoint; compiled into production output
VITE_GRAPHQL_DEBUG	No	Disabled	<code>src/lib/apollo.ts</code>	Set to 1; logs endpoint and token presence only in Vite development mode
DISABLE_HMR	No	HMR enabled	<code>vite.config.ts</code>	Set to <code>true</code> to disable Vite HMR
GEMINI_API_KEY	No	Undefined	<code>vite.config.ts</code>	Injected as <code>process.env.GEMINI_API_KEY</code> but unused by current source

All frontend-injected values are observable by browser users. Do not put private secrets in `VITE_*` values or in `GEMINI_API_KEY`.

`VITE_GRAPHQL_URI` is resolved when the dev server starts or the bundle is built. Restart/rebuild after changing it.

7.2 Test variables

Variable	Required for	Default
SUPREME_E2E_USERNAME	Real-backend integration smoke	None
SUPREME_E2E_PASSWORD	Real-backend integration smoke	None
SUPREME_E2E_GRAPHQL_URI	Integration endpoint override	<code>http://localhost:8000/graphql/</code>
CI	CI-specific Playwright controls	Unset locally

When `CI` is truthy, Playwright:

- rejects `focused test.only`;
- retries failures twice;
- starts a fresh Vite server instead of reusing an existing server.

Both Playwright configurations set one worker and disable full parallel execution.

7.3 Cross-origin backend requirements

Apollo sends:

- `credentials: "include";`
- normal GraphQL HTTP headers;
- `X-SuperAdmin-Authorization: <raw-token>` when authenticated.

The token does not use a `Bearer` prefix. For cross-origin deployments, backend CORS must permit:

- the exact frontend origin;
- credentialed requests;
- the custom `X-SuperAdmin-Authorization` header;
- required GraphQL methods and content headers.

8. Commands and local workflow

Command	Purpose
<code>npm run dev</code>	Start Vite on <code>0.0.0.0:3001</code>
<code>npm run build</code>	Produce optimized static files in <code>dist/</code>
<code>npm run preview</code>	Serve the production build on <code>0.0.0.0:3001</code>
<code>npm run lint</code>	Run <code>tsc --noEmit</code>
<code>npm run test:mocked</code>	Run self-contained mocked E2E tests
<code>npm run test:integration</code>	Run the real-backend read-only smoke
<code>npm run test:e2e</code>	Run mocked and real-backend E2E projects
<code>npm run test:ct</code>	Run React component tests
<code>npm test</code>	Run E2E projects, then component tests
<code>npm run test:ui</code>	Open mocked E2E tests in Playwright UI
<code>npm run test:debug</code>	Debug mocked E2E tests
<code>npm run test:ct:ui</code>	Open component tests in Playwright UI
<code>npm run test:all</code>	Alias for <code>npm test</code>

Recommended routine before review:

```
SH
npm run lint
npm run build
npm run test:mocked
npm run test:ct
```

Run the integration smoke when a compatible backend and dedicated credentials are available.

9. Authentication and session lifecycle

9.1 Login contract

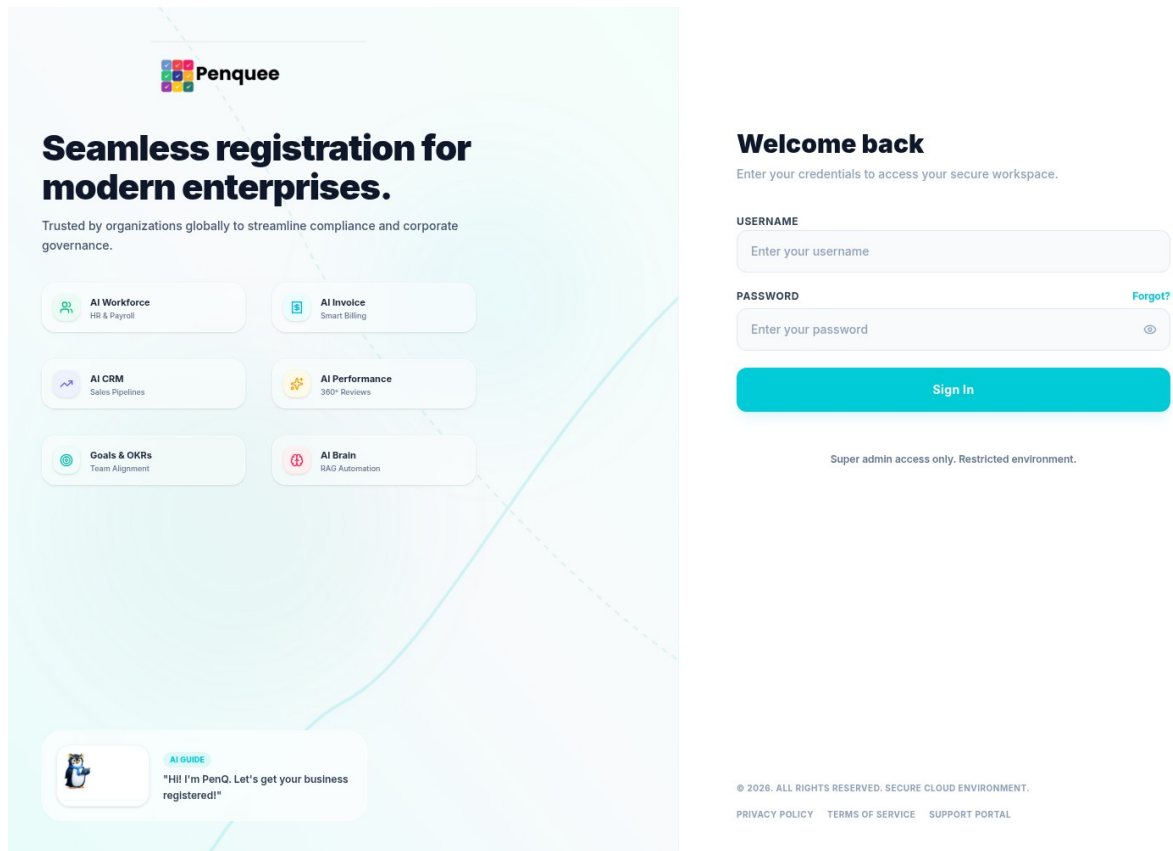


Figure 2. Login shell rendered by `App` before an authenticated admin session is mounted.

App.tsx calls SuperAdminLogin with a trimmed username and password. A login is accepted only when the response includes:

- success: true;
- a non-empty token;
- a superAdmin object;
- superAdmin.isActive: true.

Domain failures display the backend message where available. Transport or unexpected errors display the thrown message.

9.2 Stored session

The app stores three values in window.sessionStorage:

Key	Content
token	Raw super-admin token
adminUsername	Username shown in the shell
adminExpiresAt	Optional backend-provided expiration timestamp

Lifecycle:

TEXT

App starts

```

├─ missing token/username → login shell
├─ token + username + valid/no expiry → dashboard
├─ token + username + invalid/past expiry
  → clear keys → login shell with expiry message

```

Successful login

```

→ write token and username
→ write or remove expiresAt
→ mount AdminDashboard

```

Expiry timer or recognized auth error

```

→ clear all keys
→ dispatch/listen for admin-session-expired
→ return to login shell

```

Manual logout

```

→ clear all keys
→ return to login shell

```

sessionStorage is tab-scoped. A separately opened tab may not share the session. Browser-side session state is a convenience only; every backend admin resolver must enforce authorization.

9.3 Expiration handling

App performs three expiry checks:

1. startup validation of `adminExpiresAt`;
2. a timeout scheduled for the remaining duration;
3. a listener for `admin-session-expired` emitted by Apollo error handling.

An invalid timestamp is treated as expired at startup. If no expiry is provided, the app does not schedule an expiry timeout. `AdminLayout` still calculates `sessionRemaining` and sets it to `00:00:00`, but the current JSX never renders that state.

10. Apollo transport, cache, and error handling

10.1 Link chain

The Apollo client composes:

TEXT

```
errorLink → authLink → HttpLink
```

- `errorLink` detects authentication failures and clears the session.
- `authLink` reads the latest token for each operation and adds the raw custom header.
- `HttpLink` sends the request to `VITE_GRAPHQL_URI` with cookies enabled.

10.2 Cache and fetch defaults

TEXT

```

InMemoryCache
  resultCaching: false

watchQuery
  fetchPolicy: cache-and-network
  errorPolicy: all

query
  fetchPolicy: cache-first
  errorPolicy: all

```

Page-level overrides include:

- plans, promo codes, companies, expenses, and expense detail use `cache-and-network`;
- selected-user activity uses `network-only`;
- mutations commonly call `refetch` explicitly after success.

Disabling result caching can increase rendering and normalization work. Change it only after validating pages that transform result objects locally.

10.3 Authentication error recognition

The client logs out when:

- the network status is HTTP 401 or 403; or
- a GraphQL/network error message contains `expired`, `unauthorized`, `unauthenticated`, `authentication credentials`, or `invalid token`.

Backend auth errors should use a compatible status or message if automatic logout is expected. Domain authorization failures that should not terminate the session need distinguishable messages/status codes.

10.4 Error presentation

Pages generally expose loading, error, empty, and populated states. Error handling is not centralized beyond auth expiration:

- Dashboard renders a detailed error panel and logs to the console only in development.
- Other pages display inline error text or a toast for mutation failures.
- Apollo `errorPolicy`: `"all"` permits partial data alongside GraphQL errors, so page code must not assume `data` implies a fully successful response.
- Plans intentionally show canonical Free/Premium fallback cards when the plans query fails or returns no records. The fallback is display-only and does not make save/delete work offline.

Several current mutation handlers treat a resolved request as success without consistently checking a returned `{ success, message }` domain payload. Promo-code handlers are stricter than some plan, company, subscription, user, and expense handlers. Treat this as a hardening item: every mutation should validate the domain outcome before showing success or refetching.

11. GraphQL operation map

`src/lib/graphql.ts` is the executable frontend selection-set source. TypeScript interfaces in pages document how results are consumed, but they do not validate the backend schema at compile time.

11.1 Operations consumed by mounted UI

Runtime area	Queries	Mutations
Login shell	—	<code>SuperAdminLogin</code>
Dashboard	<code>AdminRevenueSummary</code>	—
Plans tab	<code>AdminPlans</code>	<code>AdminSavePlan</code> , <code>AdminDeletePlan</code>
Promo Codes tab	<code>AdminPromoCodes</code>	<code>AdminSavePromoCode</code> , <code>AdminDeactivatePromoCode</code>
Companies list/360	<code>AdminCompanies</code> , <code>AdminCompanyPaymentHistory</code>	<code>AdminUpdateCompanyDetail</code> , <code>AdminAssignCompanyPlan</code> , <code>AdminUpdatePaymentStatus</code> , <code>AdminRequestRefund</code> , <code>AdminManualSubscriptionAction</code>

Runtime area	Queries	Mutations
Subscriptions	AdminPayments, AdminCompanies	AdminUpdatePaymentStatus, AdminUpdateCompanySubscription, AdminSuspendCompany, AdminResumeCompany, AdminManualSubscriptionAction
Users	AdminUsers, AdminCompanies, AdminUserActivityLogs	AdminUpdateUser
Accounts → Overview	AdminInfrastructure	—
Accounts → Expenses	AdminExpenses, AdminExpenseById	AdminCreateExpense, AdminUpdateExpense, AdminSetExpenseStatus
Accounts → Monthly/Annual	AdminAccountsReport	—

11.2 Defined but unused operations

These documents are exported from `src/lib/graphql.ts` but have no consumer in current application source:

Operation	Intended domain	Current status
AdminModules	Module/package list	Defined, not called
AdminModuleDetail	Module/package detail	Defined, not called
AdminSaveModule	Module/package persistence	Defined, not called
AdminDeleteModule	Module/package deletion	Defined, not called
AdminSetModuleOffer	Module/package offer	Defined, not called
AdminPlanDetail	Plan detail	Defined, not called
AdminDeactivatePlan	Plan deactivation	Defined, not called

Do not infer that these workflows are available merely because a document exists. Before adopting one, reconcile its variables and selection set with the current backend schema, add a UI consumer, and add tests.

11.3 Operation ownership rules

When changing a contract:

1. Update the GraphQL document.
2. Update the page result/variable interfaces.
3. Update mocked E2E route fixtures.
4. Update component-test mocks/harnesses.
5. Confirm refetch variables and mutation result handling.
6. Confirm backend authorization, validation, audit, and error behavior.
7. Run mocked and component suites, then the real-backend smoke when possible.

12. Data flow and backend contract

12.1 Standard request flow

TEXT

```
User interaction
  → page-local state/validation
  → Apollo useQuery, useLazyQuery, or useMutation
  → errorLink
  → authLink reads current session token
  → HttpLink sends HTTP GraphQL request
  → external service authorizes and executes
  → Apollo result
  → page transforms/filters/sorts data
  → card, table, dialog, report, or toast
```

12.2 Query and mutation patterns

- List pages translate local controls into GraphQL variables.
- Some filters are server-side; many column filters operate only on the returned records.
- Expenses use server-side search, sort, limit, and offset, then apply column filters to the loaded page.
- Companies request payment history only when a company is selected.
- Users load activity lazily for the selected user and perform additional client-side scoping.
- Mutations use client validation for usability, inspect success/message, then refetch relevant data.

The last behavior is the intended contract, not yet a universal implementation guarantee. Review each handler when touching it; a GraphQL request can resolve successfully while its domain payload reports `success: false`.

12.3 Data conventions

Convention	Frontend expectation
Money	Integer cents; convert only at the input/display boundary
Currency	Uppercase code passed to <code>Intl.NumberFormat</code> where available
Date/time	JavaScript-compatible date strings
Mutation outcome	Accurate success and message fields
Activity payload	May be scalar, structured, or serialized JSON
Audit reasons	Collected by sensitive flows; backend must persist and enforce
Authorization	Super-admin guard on every admin query and mutation

`admin-utils.ts` supplies money conversion, localized date display, `datetime-local` conversion, tolerant module parsing, and JSON formatting.

12.4 Client validation versus authority

Client validation improves feedback but is not authoritative. The backend must validate:

- permissions and tenant boundaries;
- plan, promotion, company, and user changes;
- payment state and refund amounts;

- manual subscription actions;
- expense totals, line items, and allowed status transitions;
- audit reason presence and audit-record creation;
- idempotency and concurrency where financial actions are involved.

13. Mounted modules and shared components

13.1 Mounted destination map

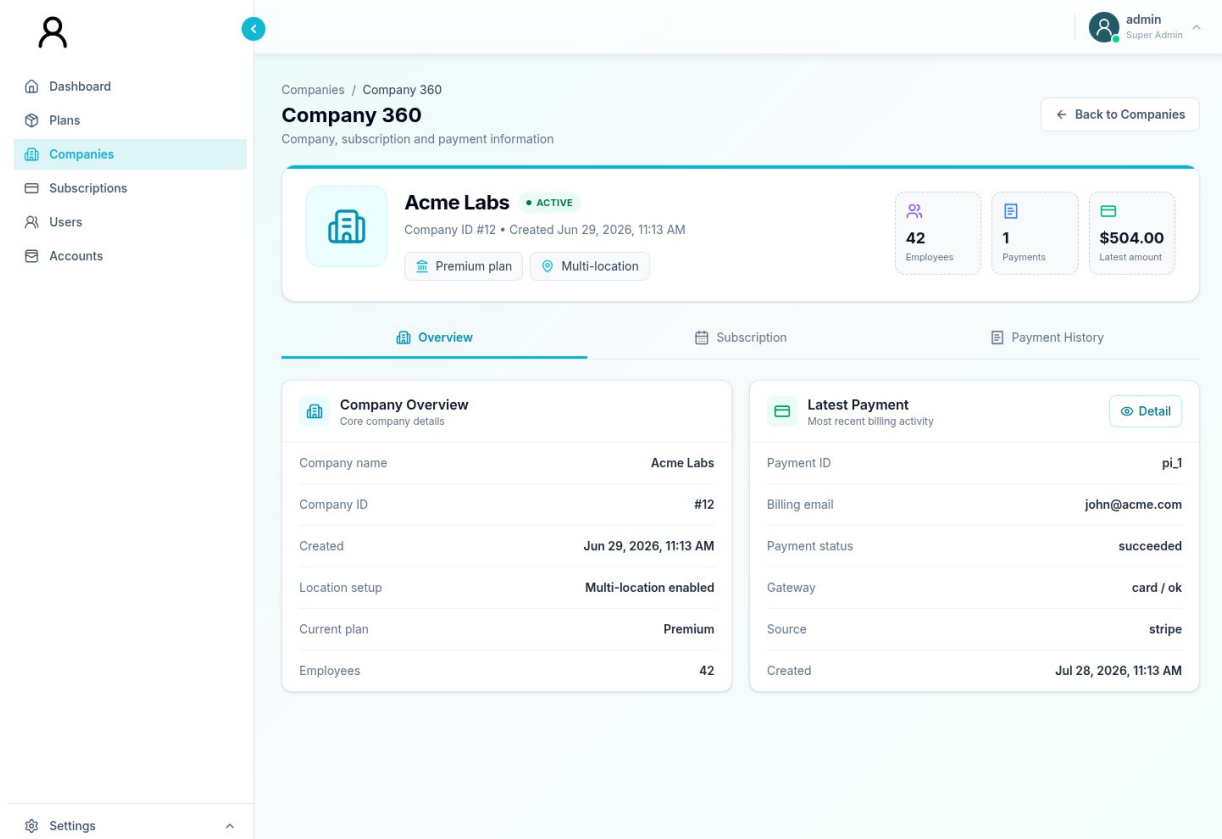


Figure 3. Representative nested module: Company 360 rendered within the Companies destination.

AdminDashboard directly mounts six page components:

Page component	Mounted as	Notes for maintainers
Dashboard	Dashboard	Derives net revenue when backend omits it
Plans	Plans	Contains Plans and Promo Codes tabs; exports Packages alias
Companies	Companies	Contains Company 360 and payment-history detail
Subscriptions	Subscriptions	Combines payment data with company display data
Users	Users	Includes per-user activity dialog
Accounts	Accounts	Mounts Infrastructure and Expenses internally

13.2 Accounts internal composition

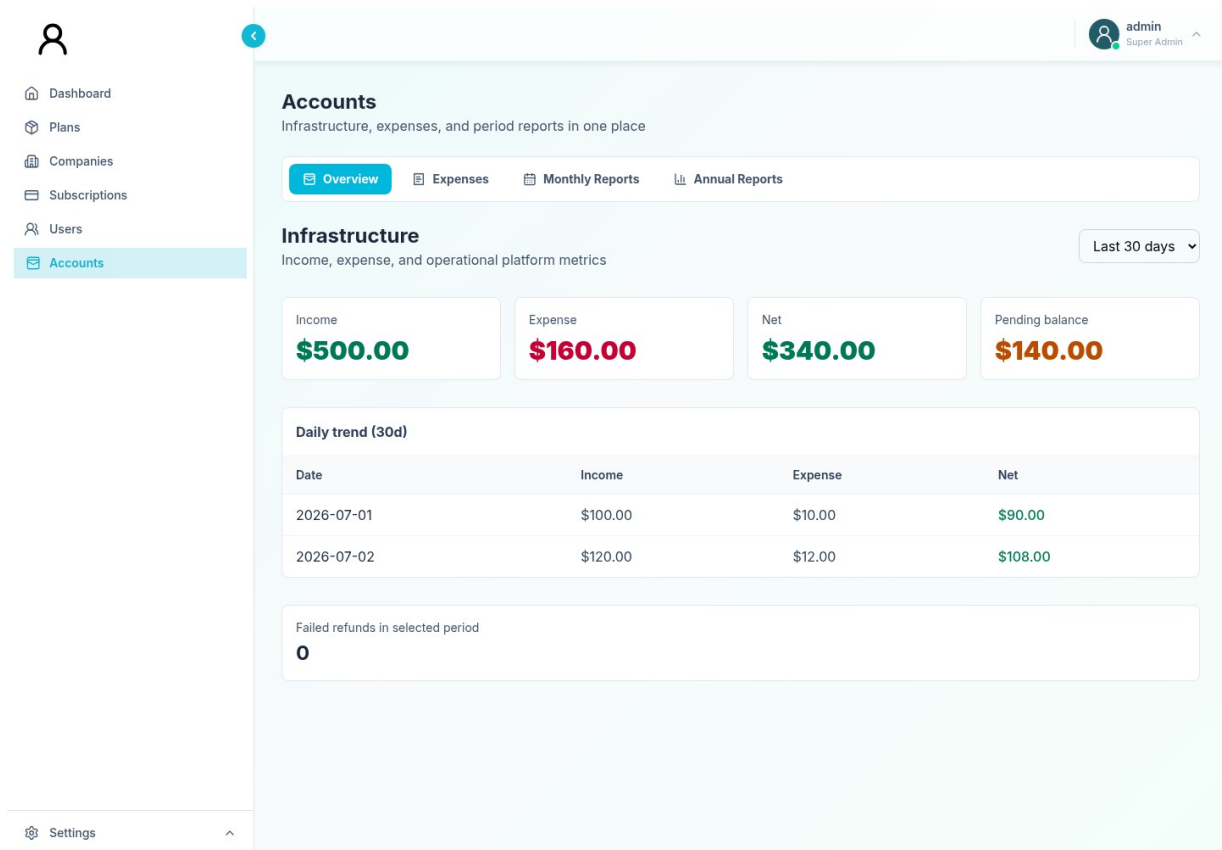


Figure 4. Accounts internal tab composition with Infrastructure mounted as Overview.

Accounts.tsx owns four tabs:

TEXT

```
overview → Infrastructure
expenses → Expenses
monthly → AccountsReportTable(period="monthly")
annual → AccountsReportTable(period="annual")
```

Infrastructure and Expenses are therefore reachable, but not top-level navigation pages.

13.3 Shared components

Component	Purpose	Important behavior
AdminLayout	Responsive shell	Desktop collapse, mobile overlay, focus trap/restoration, Escape handling, menus, unrendered countdown calculation
ColumnFilter	Multi-select table filter	Normalizes empty values, counts/sorts options, searches, portals popup, closes on Escape/outside click
DateRangePicker	Dual-month range selector	Presets; currently used by Company 360 payment history
Toast	Global transient feedback	Success/error/info variants, four-second default, manual close
useDebounceValue	Input stabilization	Default 300 ms delay

Most `ColumnFilter` consumers filter only records already returned by the backend; they do not automatically search unrequested server pages.

13.4 Export and print behavior

Plans, Companies, Subscriptions, and Accounts create spreadsheet-compatible downloads with browser `Blob/object` URLs. Print/PDF controls open a new window and write escaped tabular HTML for browser printing. “PDF” is not native PDF generation; the operator must choose **Save as PDF** in the browser.

Pop-up blockers can prevent print flows. Changes to export code should preserve HTML escaping for user/backend-provided text.

14. Unmounted components and unused operations

14.1 Standalone User Activity Log

`src/components/admin/pages/UserActivityLog.tsx` implements a complete standalone audit-log view with actor, action, company, user, and date filters. It is:

- exported/tested;
- covered by component tests;
- not imported or rendered by `AdminDashboard`;
- absent from top-level navigation.

The mounted Users page has a separate per-user activity dialog that consumes the same `AdminUserActivityLogs` operation.

14.2 Unused GraphQL documents

The seven operations listed in 11.2 are not runtime capabilities. In particular:

- module/package CRUD and offer documents have no page consumer;
- plan detail is not used by the Plans implementation;
- plan deactivation is not used; the page currently uses save/delete and promo deactivation flows.

14.3 Safe treatment

Do not remove unmounted code or unused documents solely because they are unused without confirming roadmap and backend dependencies. If retained, keep them visibly classified so documentation and tests do not overstate product reachability.

15. Testing topology

15.1 Test layers

Layer	Configuration	Server/runtime	Backend dependency
Mocked E2E	<code>playwright.config.ts</code> , <code>project mocked-chromium</code>	Vite on port 3001, Google Chrome	No; GraphQL routes are mocked
Real-backend smoke	Same config, project <code>real-backend-chromium</code>	Vite on port 3001, Google Chrome	Yes; read-only backend and credentials
React component	<code>playwright-ct.config.ts</code>	CT runtime on port 3100, Playwright Chromium	No; Apollo/component harnesses

Layer	Configuration	Server/runtime	Backend dependency
Static validation	tsconfig.json	TypeScript compiler	No
Production compilation	vite.config.ts	Vite/Rollup	No

Verified test discovery:

- 29 E2E/integration tests in 10 files: 28 mocked and 1 real-backend;
- 52 component tests in 13 files.

Use `--list` to obtain current counts after the suite changes.

15.2 Test directories

TEXT

```
tests/
├── auth/
├── dashboard/
├── layout/
├── commerce/
├── accounts/
├── users/
├── integration/
├── fixtures/

component-tests/
├── commerce/
├── operations/
├── shared/
├── fixtures/
```

The real-backend smoke is deliberately read-only: it logs in, verifies dashboard data, and navigates existing pages. It must not create, update, refund, suspend, or delete production-like records.

15.3 Targeted commands

SH

```
# Discover tests without running them
npx playwright test --list
npx playwright test --config=playwright-ct.config.ts --list

# One mocked E2E file
npx playwright test tests/auth/authentication.spec.ts \
  --project=mocked-chromium

# Tests matching a title
npx playwright test --project=mocked-chromium \
  --grep="password visibility"

# One component test file
npx playwright test component-tests/commerce/plans.spec.tsx \
  --config=playwright-ct.config.ts

# Real-backend read-only smoke
SUPREME_E2E_USERNAME=admin@example.com \
SUPREME_E2E_PASSWORD=secret \
SUPREME_E2E_GRAPHQL_URI=http://localhost:8000/graphql/ \
npm run test:integration
```

15.4 Generated artifacts

Playwright writes to:

- `playwright-report/`;

- test-results/;
- playwright/.cache/.

These paths are generated and ignored by Git.

15.5 Change-to-test map

Change area	Minimum focused validation
Login/session/Apollo auth	Auth E2E + app/provider component tests
Navigation/layout/accessibility behavior	Layout E2E + admin-layout component tests
Plans/promotions	Plans E2E + plans component tests
Companies/payment history	Companies E2E + companies component tests
Subscriptions	Subscriptions E2E + subscriptions component tests
Users/activity	Users E2E + users/user-activity component tests
Accounts/reports	Accounts E2E + accounts/infrastructure component tests
Expenses	Expenses E2E + expenses component tests
GraphQL documents	Owning feature tests + real-backend smoke
Build/config/dependencies	Type validation + production build + relevant test layer

16. Build and deployment

16.1 Production build

```
SH
npm ci
VITE_GRAPHQL_URI=https://api.example.com/graphql/ npm run build
```

Deploy the generated `dist/` directory to a static host.

The current production build succeeds but emits a chunk-size warning: the main minified JavaScript bundle is larger than Vite's 500 kB warning threshold. This is a performance signal, not a build failure.

16.2 Pre-deployment verification

```
SH
npm run lint
npm run build
npm run preview
```

Then verify in a browser:

- login against the intended backend;
- authenticated query headers and CORS;
- expiry and logout;
- every mounted destination;
- sensitive actions in a safe test environment;
- download and print pop-ups;
- responsive navigation.

16.3 Hosting requirements

- Serve over HTTPS.
- Set the final GraphQL endpoint before building.
- Allow the deployed origin and custom auth header in backend CORS.
- Configure credentialed cookies correctly if the backend uses them.
- Never inject backend secrets into the frontend.
- Serve `dist/index.html` and its hashed assets with correct MIME types.
- Use appropriate caching: long-lived immutable caching for hashed assets and conservative caching for `index.html`.

The current state-based navigation has no deep URL paths, so provider-specific route rewrites are not required. Revisit this if a router is added.

Vite `base` is unset and branding assets use root-absolute `/assets/...` paths. Root hosting is the supported default. A subpath deployment requires a deliberate Vite `base` setting and asset-path review.

16.4 Security headers and external resources

The repository does not define a Content Security Policy. If the hosting layer adds one, account for:

- the GraphQL origin in `connect-src`;
- Google Fonts stylesheet/font origins, or self-host Inter;
- same-origin images;
- Vite-generated scripts and styles;
- browser print windows.

Prefer self-hosting Inter for offline or strict-CSP deployments.

17. Customization and extension workflow

17.1 Branding and visual system

Brand assets live in `public/assets/`:

- `penquee_logo.png`;
- `penquee_mascot.png`;
- `penquee-favicon.svg`.

Login marketing content is in `App.tsx`; the document title and favicon reference are in `index.html`. Global font and scrollbar rules are in `src/index.css`. Most feature colors and spacing are Tailwind classes inside `TSX`.

17.2 Add a mounted top-level page

1. Create the page component in `src/components/admin/pages/`.
2. Add required GraphQL documents to `src/lib/graphql.ts`.
3. Extend `CurrentPage` in `AdminDashboard.tsx`.
4. Add the render branch in `AdminDashboard`.
5. Extend the page union and `navItems` in `AdminLayout.tsx`.
6. Add mocked GraphQL fixtures.
7. Add component coverage for loading, error, empty, populated, and interaction states.
8. Add E2E navigation and critical-flow coverage.

9. Validate keyboard/focus behavior on mobile and desktop.

Because page types are duplicated, a refactor to a shared page ID type/config is a sensible precursor to frequent navigation changes.

17.3 Add or change a GraphQL operation

1. Confirm the backend schema, authorization, and error contract.
2. Add/update the `gql` document.
3. Define result and variable types near the consumer.
4. Decide whether the operation needs `cache-first`, `cache-and-network`, or `network-only`.
5. Handle loading, partial data/error, empty, and retry/refetch states.
6. For mutations, validate `success`, `surface message`, and refresh affected data.
7. Update mocked and component fixtures.
8. Run the real-backend smoke against the integrated schema.

There is no generated GraphQL typing. Handwritten types can drift silently; consider schema-based code generation if the contract grows.

17.4 Extend expense status behavior

The UI exposes transitions:

From	Allowed destinations
Draft	Pending, Rejected, Archived
Pending	Approved, Rejected, Archived
Approved	Paid, Archived
Paid	Archived
Rejected	Draft, Archived
Archived	Draft

Update UI transition options, backend domain rules, audit records, mocked fixtures, and component/E2E tests together. The backend is authoritative.

17.5 Change transport/session behavior

Edit `src/lib/apollo.ts` for endpoint/header/cache/error-link changes and `App.tsx` for session semantics. Keep synchronized:

- backend token format and CORS;
- storage key migration;
- expiry behavior;
- automatic logout criteria;
- unrendered layout countdown state and its test expectations;
- authentication tests.

18. Troubleshooting

18.1 GraphQL is unreachable

Check:

1. VITE_GRAPHQL_URI was set before server start/build.
2. The endpoint path and trailing slash match the backend.
3. The endpoint is reachable from the browser, not just the frontend host.
4. CORS permits the frontend origin, credentials, and custom header.
5. Browser Network tools distinguish CORS, transport, HTTP, and GraphQL failures.

Development diagnostics:

```
SH
```

```
VITE_GRAPHQL_DEBUG=1 npm run dev
```

This logs endpoint and token presence, never the token value.

18.2 Login immediately returns to sign-in

- Confirm `expiresAt` is a valid future timestamp.
- Inspect HTTP 401/403 and GraphQL auth messages after login.
- Check the three `sessionStorage` keys.
- Confirm the backend expects the raw custom header, not `Bearer <token>`.
- Verify the super-admin record is active.

18.3 Page is empty or missing fields

- Compare the operation selection set with the backend schema/resolver.
- Compare page interfaces with returned shapes.
- Inspect `backend-change-requests.md` for requested but potentially unimplemented capabilities.
- Remember the repository has no backend.
- Treat Plans fallback cards as display-only.

18.4 Filters miss records

Many column filters apply only to the current query result. Expenses filter the current server page after server-side pagination. Implement equivalent backend filter arguments for global searching/filtering.

18.5 Full tests stop before component tests

`npm test` executes E2E first, including the real-backend project. Supply credentials and a reachable service, or run:

```
SH
```

```
npm run test:mocked
npm run test:ct
```

18.6 Browser launch fails

```
SH
```

```
npx playwright install chrome chromium
```

Install Playwright OS dependencies on clean Linux agents if required.

18.7 Print/PDF action appears inert

Allow pop-ups. The application opens a new window and relies on the browser print dialog. There is no native application-side PDF renderer.

18.8 Font or styling differs in restricted networks

Inter loads from Google Fonts. A strict CSP or offline environment can block it; system sans-serif remains as a fallback. Self-host the font for deterministic rendering.

18.9 Production build warns about chunk size

The app currently ships a large main bundle. Measure before optimizing, then consider:

- dynamic imports for page components;
- route/page-level lazy loading;
- Rollup `manualChunks`;
- auditing large dependencies.

Do not simply raise the warning threshold without documenting the performance decision.

19. Security and maintenance

19.1 Security invariants

- Enforce super-admin authorization on every backend operation.
- Treat the session token as sensitive; never log or export it.
- Use HTTPS for frontend and API.
- Restrict CORS to approved origins.
- Never place secrets in Vite-injected variables.
- Enforce payment, refund, subscription, user, and expense rules on the backend.
- Persist actor, reason, target, before/after values, and timestamp for sensitive mutations.
- Sanitize/escape values embedded in print/export HTML.
- Review third-party dependencies and lockfile changes.

`sessionStorage` reduces persistence compared with `localStorage`, but any script executing in the origin can access it. XSS prevention and a restrictive CSP remain important.

19.2 Maintenance cadence

For routine changes:

1. Keep GraphQL documents, handwritten types, backend schema, and test fixtures aligned.
2. Run static validation and production build.
3. Run focused feature tests plus mocked/component suites.
4. Run the integration smoke for contract-affecting changes.
5. Review browser console/network output.
6. Review build size and dependency changes.
7. Update both developer and user manuals when reachability or behavior changes.

19.3 Financial correctness

- Keep money as integer cents across input conversion, GraphQL, calculations, exports, and storage.

- Avoid floating-point arithmetic for stored financial values.
- Treat client-derived totals as presentation unless the backend confirms them.
- Make refund and manual-payment endpoints idempotent or otherwise safe against retry.
- Enforce allowed expense transitions server-side.
- Derive audit actors from the authenticated backend principal. Expense status calls currently submit a client-controlled actor and may submit an empty reason; neither is trustworthy evidence.
- Validate, size-limit, and redact manual payment/subscription JSON payloads on the backend.
- Enforce duplicate/idempotency rules server-side. Expense duplicate checks cover only the currently loaded page.

19.4 Auditability

Audit-sensitive workflows include:

- plan and promo changes;
- company detail and plan changes;
- payment status and refund requests;
- subscription due-date, suspension, resume, and manual processing;
- user role/active state changes;
- expense creation, update, and status change.

Backend logging is mandatory even where the UI collects a reason.

20. Known limitations and technical debt

Limitation	Engineering impact
Frontend-only repository	Cannot run live workflows without a compatible external service
No URL router	No deep links, browser history, or persisted destination
Duplicated page ID union	Layout and dashboard can drift
Handwritten GraphQL result types	Schema/type drift is detected only at runtime/tests
Inconsistent domain-success checks	Some resolved mutations can show success even when a payload reports failure
Unused GraphQL documents	Source can overstate available UI capability
Standalone activity page unmounted	Tested code is not operator-reachable
Main bundle over Vite warning threshold	Potential startup/performance cost
No ESLint/formatter	Style and some defect classes lack automated enforcement
No automated accessibility audit	Keyboard tests exist, but broader WCAG regressions are possible
No visual-regression suite	Styling regressions rely on functional tests/manual review
No CI/CD or deployment manifests	Release process is environment-specific and manual
No CSP in repository	Hosting layer must define and test it
External Google Font	Offline/CSP rendering can differ

Limitation	Engineering impact
Placeholder footer/login links	Forgot, Privacy, Terms, and Support point to #
Client-side column filters	Results may represent only the loaded backend page
Client-only duplicate detection	Expense duplicates outside the loaded page are not detected
Browser-dependent PDF workflow	Pop-ups and print settings affect exports
Plans fallback data	May conceal backend plan-query failure if not monitored
No Node version pin	Local/CI environments can diverge
No production environment template	Configuration conventions are documentation-dependent
Session countdown is calculated but not rendered	Operators cannot see remaining session time; the component-test title/assertion for a live countdown is stale and should be corrected with the UI

Also note:

- GEMINI_API_KEY is injected but unused and should not hold a secret.
- Apollo cache resultCaching is disabled.
- The app has no application-level error boundary.
- Some expense audit fields are client-controlled; the backend must derive actor identity and enforce reasons.
- Mutation authorization and business integrity cannot be secured in the browser.

Appendix A. Evidence baseline

This manual is grounded in:

- package.json and package-lock.json;
- vite.config.ts, tsconfig.json, and index.html;
- playwright.config.ts and playwright-ct.config.ts;
- src/main.tsx, src/App.tsx, and src/providers/GraphQLProvider.tsx;
- src/lib/apollo.ts, src/lib/graphql.ts, and src/lib/admin-utils.ts;
- src/components/admin/ and all page components;
- tests/ and component-tests/;
- TESTING.md, tests/README.md, and backend contract notes.

Verified against the current source snapshot:

- TypeScript validation passes.
- The Vite production build passes.
- E2E discovery reports 29 tests in 10 files.
- Component discovery reports 52 tests in 13 files.
- Six top-level pages are mounted.
- Infrastructure and Expenses are reachable through Accounts tabs.
- Standalone User Activity Log is not mounted.
- Seven exported GraphQL documents are currently unused.

Appendix B. Release checklist

Code and contract

- ☐ GraphQL documents match the intended backend schema.
- ☐ Handwritten result/variable types and fixtures are updated.
- ☐ Sensitive mutations enforce backend authorization, validation, and audit.
- ☐ Money remains integer cents.

Validation

- ☐ `npm ci`
- ☐ `npm run lint`
- ☐ `npm run build`
- ☐ `npm run test:mocked`
- ☐ `npm run test:ct`
- ☐ Real-backend smoke with dedicated credentials, when available

Deployment

- ☐ Final `VITE_GRAPHQL_URI` set before build.
- ☐ HTTPS enabled.
- ☐ CORS permits only approved origins and required headers.
- ☐ Cookie attributes verified, if used.
- ☐ No frontend-injected secrets.
- ☐ `index.html` and hashed-asset cache rules configured.
- ☐ CSP, font loading, exports, and pop-ups verified.

Product integrity

- ☐ All six mounted destinations load.
- ☐ Session restoration, expiry, and logout work.
- ☐ Loading, error, empty, and populated states are verified.
- ☐ Mobile navigation and keyboard focus behavior are verified.
- ☐ User-facing manual updated if reachable workflows changed.